Project 1: Where in the World?

Design Review Cycle — Guess the City / 20 Questions

EECS 498-008 — Amit Sail (amitsail) — 2026-09-29

Honor Code Pledge

> "I HAVE received aid from an LLM or automated coding assistant on this submitted work. If I received aid, the specific tool used was [claude-sonnet-5]. Regardless of assistance, I fully understand and can defend all code and design choices within this submission, and I have not concealed any violations of the Honor Code."

Signed: AMIT SAIL UMID: 4809 0678 Date: 2026-09-29

Context

HW1's design document scored 100/100, with specific written feedback for Project1: work out the design before describing what was built, keep the document to design reasoning rather than documenting existing tutorial code, and resolve one open question directly — how the mystery answer can stay fixed for an entire game without ever being shown to the player, for both reasoning and non-reasoning models. HW1's Goldfish review had also reviewed finished code instead of a design, which the grader flagged as backwards. This document redoes that review cycle correctly: a design is proposed first, a second independent reviewer critiques only that design with no access to any code, and the design is revised in response — before any of it was implemented.

Prompt to Elephant

Verbatim prompt addressing the HW1 feedback above.

We got 100/100 on HW1's design document, but the grader left specific notes to address for Project1:

1. The UI Mockups section presented finished-app screenshots and called them mockups; a design document is meant to work out the design prior to implementation, not document what was already built. 2. The Frontend Code Structure and Backend Code Structure sections were call-graph documentation of existing tutorial code. The grader's note: this doesn't belong in a design document, or at most deserves one summarizing sentence, since the architecture is inherited unchanged from the tutorials. 3. Section 5 (Game Design & Infrastructure Reuse) was flagged as exactly the right level of description - prose, with choices and justification. 4. The open design question the grader posed directly: "The mystery city must be fixed at the start of the game so that subsequent turns can be consistent, but the city cannot be shown to the user! How to do that?" This lines up with the official HW1 General Feedback item #2 (the design must support both reasoning and non-reasoning models, and the answer must stay fixed) and item #3 (don't over-engineer serialization of Send/Clear or block game resumability). The design has to solve this within those constraints, and without changing /llmchat's data format. 5. Our HW1 Goldfish review critiqued the finished CODE, which the grader flagged as backwards - Goldfish is supposed to critique the DESIGN summary, not an implementation.

Please draft the design that solves point 4, addressing points 1-3 along the way. Prose design reasoning only - no code, no call graphs of existing tutorial structure.

Elephant Design Summary

No code. Response to the prompt above.

Architecture: unchanged from the tutorial. The frontend keeps the SwiftUI Rein/Chat/AppViewModel shape from llmChat; the backend keeps the Starlette main.py/handlers.py/runners.py split. The game layer introduces no new structural abstractions — only new call sites into the existing pieces, plus one new route (/llmclear) that reuses the existing OpenAIRequest format rather than inventing a new one, per the assignment's constraint.

Screen design: the single screen needs four elements the spec requires — a timeline of prompt-completion turns with tokens shown as they stream in, independent optional system/user prompt fields, a Send control whose availability reflects whether either field has content, and a Clear control. A segmented picker across the top selects which game is active; switching it is treated as equivalent to pressing Clear, since the assignment's second-game requirement is about reusing infrastructure, not building a second feature surface.

The mystery-city problem: /llmchat is stateless per request but stateful per conversation — the backend replays the full stored history (system, user, and assistant turns) on every turn, so anything the model has already said is available to it again on the next turn. That's the only consistency mechanism available, and the design has to work within it without any new backend data format. It's used two ways:

First, for the choice itself: the system prompt instructs the model to silently pick one answer immediately, at the very first turn, and to treat every one of its own later replies as constrained by whatever it has already revealed in the replayed transcript — i.e., the model's own prior clues become its memory of its own choice. This works identically for reasoning and non-reasoning models, because it depends only on the visible conversation transcript, not on any special reasoning capability.

Second, for reasoning models specifically: a reasoning-capable model's hidden chain-of-thought is exactly where it decides which answer to commit to — and if that reasoning is ever shown to the player, the secret leaks before the game even starts. So the reasoning channel must never be rendered to the player while a game is active. This doesn't cost anything: the backend already persists a model's reasoning independently of whatever the client chooses to display, purely to reconstruct history for replay — so hiding it from the UI doesn't remove it from the model's own memory of its choice on the next turn. The two mechanisms compound: a reasoning model gets a private, persisted chain-of-thought reinforcing its choice on top of the same visible-transcript consistency every model gets.

Nothing here requires per-request serialization or blocking game resumability — the assignment's feedback explicitly warns against both, and this design doesn't add either: Send and Clear are not artificially serialized against each other, and a relaunched app can always continue an existing server-side history rather than being forced to start over.

New Prompt to Goldfish

Sent to a fresh model instance with no memory of the conversation above and no code or repository access — only the Elephant design summary text, reproduced verbatim in place of the placeholder below.

You are reviewing a game design proposal cold, with no other context. Do NOT read any files, do NOT explore any repository or codebase, do NOT look at any code - you have no filesystem access relevant to this task and should not attempt to use one. Base your critique ONLY on the design text below.

Context: this is a design summary for a "Guess the City" / "20 Questions" LLM chat game feature, addressing one specific hard requirement: the game's secret answer must stay fixed for the whole session and must never be revealed to the player before the reveal, for both reasoning-capable and non-reasoning LLMs, without changing the underlying chat API's data format and without over-engineering concurrency controls (the spec explicitly forbids serializing the Send/Clear buttons or blocking game resumability).

Design summary: --- [Elephant design summary - see previous section] ---

Critique this DESIGN on its own terms - is the mystery-answer-consistency mechanism actually sound? Are there gaps, edge cases, or better alternatives the design doesn't consider? Is anything over-engineered or under-engineered relative to what's actually required? Do NOT comment on code, implementation details, or file/class names - you weren't given any code, so don't invent any. This is a pure design critique, for a homework design document. Keep it under 350 words, direct and specific, not generic "best practices" filler.

Goldfish Design Critique

No code. Quoted verbatim.

Core weakness: the design conflates "commitment" with "consistency enforcement." It never pins the secret anywhere explicit; it just instructs the model to "silently pick" and then trusts that its own future replies will stay self-consistent because the transcript is replayed. Nothing forces a discrete, checkable commitment at turn 1 — a model's "silent" choice on a single forward pass can be a fuzzy distribution over plausible cities rather than one fixed fact, especially non-reasoning models with no scratch space to write it down. Drift (the model contradicting an earlier clue several turns in) is possible and the design has no detection or fallback for it.

Concrete fix that stays inside the stated constraints: have the system prompt require the turn-1 answer to be emitted inside a delimiter (e.g. a tagged token), have the backend strip that tag before showing anything to the player, store the extracted answer, and re-inject it as a hidden system-role reminder on every subsequent replayed request. This is not a new API/data format — it's just content inside the existing message roles — and needs no new concurrency control. It converts "hope the model stays consistent with its own past clues" into "the ground truth is pinned and mechanically re-asserted," which is strictly more robust and roughly the same implementation cost as what's proposed.

Second gap: the reasoning-model argument assumes persisted CoT is actually replayed back into the model's context on later turns and reinforces the choice. Many reasoning APIs don't accept prior

reasoning traces as reusable input — reasoning may be regenerated fresh each turn from the visible transcript only, or returned as an opaque non-replayable blob. If so, the claimed "compounding" benefit for reasoning models doesn't exist, though hiding the channel from the UI is still necessary regardless (defense against leaking the pick, not consistency).

Third gap: resumability is touted but the design doesn't address context-length pressure — if a long session ever gets truncated or summarized, the turn-1 commitment (the only anchor point) can be the first thing lost, silently breaking the one persistence mechanism the whole design depends on.

Send/Clear non-serialization and resumability-via-existing-history are appropriately scoped — no over-engineering there.

Elephant's Revised Design

No code. Response to the critique above.

The critique is right that "instruct the model to be consistent" isn't a real commitment mechanism — it's a hope, not a pin, and the original design has no fallback if the model drifts. Revising to a stronger version: the system prompt requires the model's very first reply to open with a tagged marker containing the answer (e.g. "[SECRET: Paris]") before any player-visible content. That marker becomes a permanent, load-bearing part of the persisted conversation transcript, replayed back to the model on every later turn exactly like every other message already is. Because /llmchat's history replay already includes every prior message verbatim, no backend change is needed to make this pin durable — it's just content inside a message the client already sends and the backend already stores.

To keep the answer hidden from the player, the client strips any leading tagged-marker span from the visible bubble before displaying it — the same mechanism already used to keep a reasoning model's chain-of-thought out of the visible timeline, extended to recognize this one additional marker. The full untouched text, including the marker, is still what gets persisted and replayed, so the model always has its own exact prior commitment to check itself against, not just an inference from its own clues.

This also resolves the critique's second concern — whether a reasoning model's chain-of-thought is actually replayed back as reusable context is now beside the point. The pin lives in ordinary visible-role content, which every model unambiguously receives back verbatim on every turn, independent of whether reasoning traces themselves are replayable.

The long-session truncation risk the critique raises is real but out of scope: /llmchat's history replay is already unbounded and untruncated in the tutorial's own design — no summarization exists anywhere in the system. This design doesn't introduce that risk, it inherits it; fixing it would mean redesigning the tutorial's persistence layer, which is outside what this assignment asks for.

Token Cost Estimate (Optional)

The Goldfish critique call above (a single design-only review request) consumed approximately 63,000 tokens end to end, dominated by the reviewing model's own reasoning rather than the ~350-word design summary it was given. The full three-step review cycle in this document (Elephant summary, Goldfish critique, revised design) is on the same order of magnitude, roughly 150,000–200,000 tokens total across all three generations.

Implementation status: the revised design — a [SECRET: <answer>] marker on the first reply, stripped client-side before display, persisted and replayed by the existing /llmchat history mechanism with no backend changes — has been implemented and verified end to end against the class harness.